How LinkedIn Scaled User Restriction System t 5 Million Queries Per Second

BYTEBYTEGO
JAN 21, 2025

Disclaimer: The details in this post have been derived from the LinkedIn Engineering Blog. credit for the technical details goes to the LinkedIn engineering team. The links to the origi articles are present in the references section at the end of the post. We've attempted to anal the details and provide our input about them. If you find any inaccuracies or omissions, ple leave a comment, and we will do our best to fix them.

One of the primary goals of LinkedIn is to provide a safe and professional environm for its members.

At the heart of this effort lies a system called CASAL.

CASAL stands for Community Abuse and Safety Application Layer. This platform i the first line of defense against bad actors and adversarial attacks. It combines technology and human expertise to identify and prevent harmful activities.

The various aspects of this system are as follows:

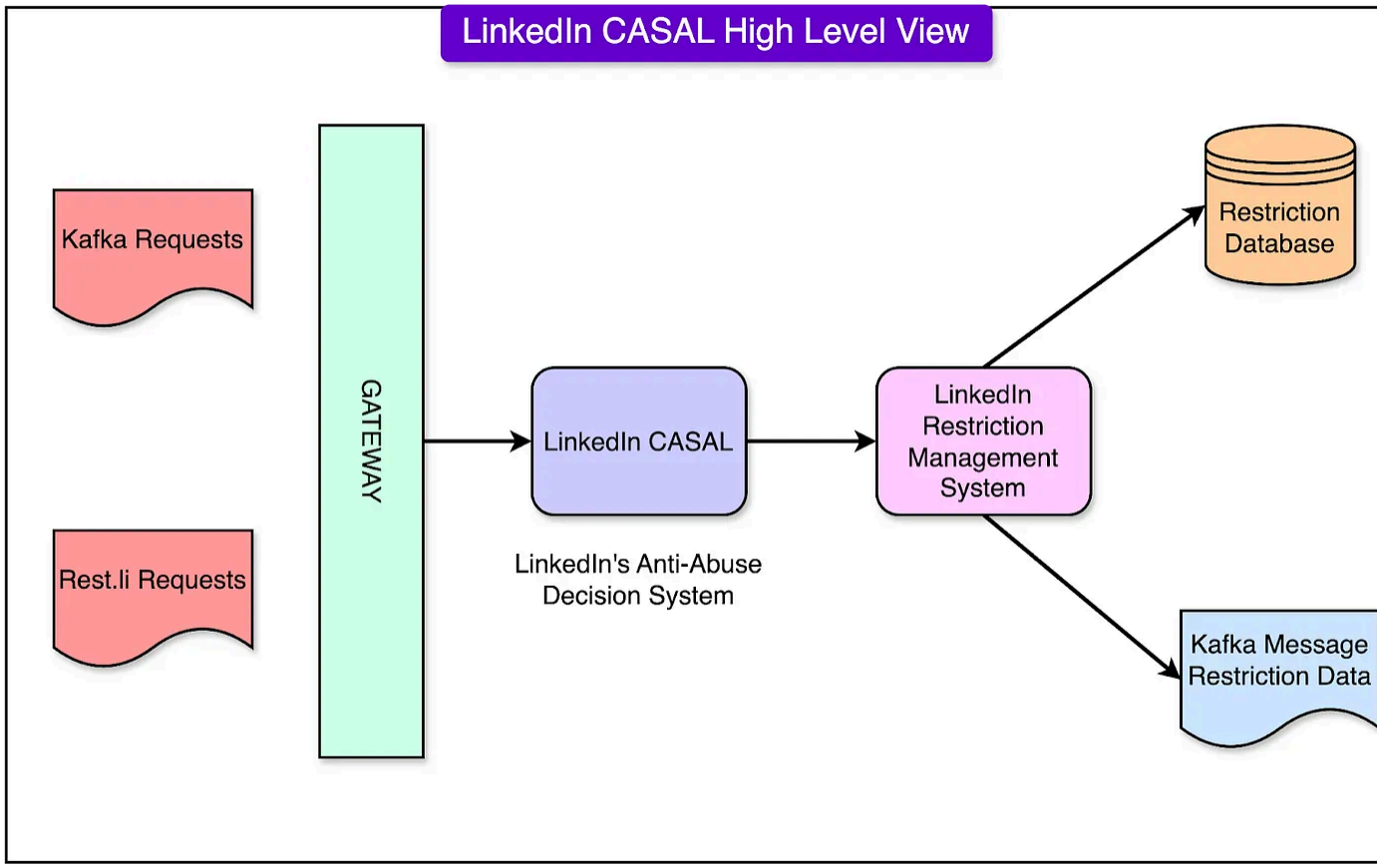
- **ML Models:** The ML models analyze patterns in user behavior to detect anythi unusual or suspicious. For example, if a user suddenly sends hundreds of connection requests to strangers or repeatedly posts harmful content, the syste can flag these activities for review.
- **Rule-Based Systems:** These systems work based on pre-defined rules. Think of them as guidelines that help the platform decide what's acceptable and what's

For instance, certain words or actions that violate LinkedIn's policies (like hate speech or spam) automatically trigger alerts.

- **Human Review Processes:** Not everything can be left to machines. A dedicated team of human experts steps in to review flagged activities and make decisions borderline cases.
- **Multi-Faceted Restrictions:** Not all harmful activities are the same. This is wh LinkedIn uses multi-faceted restrictions. Some restrictions might involve temporarily limiting a user's actions, like stopping them from sending connect requests for a while. Other situations may require more severe measures, such a permanently blocking an account if it poses a significant threat.

Together, these tools form a multi-layered shield, protecting LinkedIn's community from abuse while maintaining a professional and trusted space for networking.

In this article, we'll look at the design and evolution of LinkedIn's enforcement infrastructure in detail.



Evolution of Enforcement Infrastructure

There have been three major generations of LinkedIn's restriction enforcement system. Let's look at each generation in detail.

First Generation

Initially, LinkedIn used a relational database (Oracle) to store and manage restrictic data.

Restrictions were stored in Oracle tables, with different types of restrictions isolate into separate tables for better organization and manageability. CRUD (Create, Read Update, Delete) workflows were designed to handle the lifecycle of restriction reco ensuring proper updates and removal when necessary.

See the diagram below:

Member Id	Restriction Type 1	Restriction Type 2	.	.	TimeStamp
123	Yes	Yes			2/10/23 10:05:01 PST
456	No	No			1/10/23 13:14:01 PST

Source: LinkedIn Engineering Blog

However, this approach posed a few challenges:

- As LinkedIn grew and transitioned to a microservices architecture, the relatior database approach couldn't keep up with the increasing demand.
- The architecture became cumbersome due to Oracle's limitations in handling h query volumes and maintaining low latency.

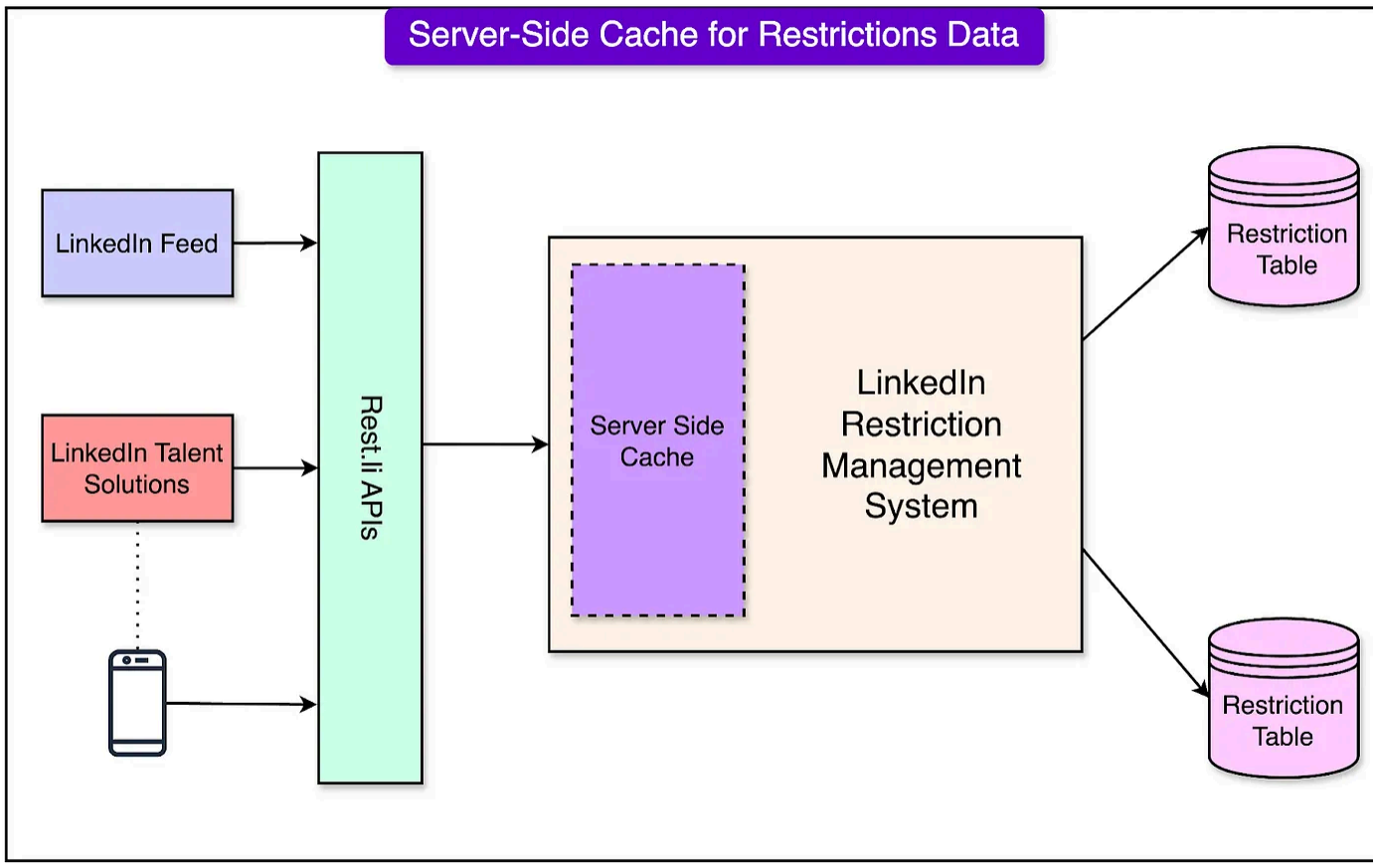
Server-Side Cache Implementation

To address the scaling challenges, the team introduced server-side caching. This significantly reduced latency by minimizing the need for frequent database queries.

A cache-aside strategy was employed that worked as follows:

- When restriction data was requested, the system first checked the in-memory cache.
- If the data was present in the cache (cache hit), it was served immediately.
- If the data was not found (cache miss), it was fetched from the database and asynchronously updated in the cache for future requests.

See the diagram below that shows the server-side cache approach:



Restrictions were assigned predefined TTL (Time-to-Live) values, ensuring the cacl data was refreshed periodically.

There were also shortcomings with this approach:

- The server-side cache wasn't distributed, meaning individual hosts had to man their caches.
- The approach worked well for low-traffic scenarios, but struggled under high cache-hit demands, necessitating further improvements.

Client-Side Cache Addition

Building on the server-side cache, LinkedIn introduced client-side caching to enha performance further. This approach enabled upstream applications (like LinkedIn

Feed and Talent Solutions) to maintain their local caches.

See the diagram below:

To facilitate this, a client-side library was developed to cache the restriction data directly on application hosts, reducing the dependency on server-side caches.

Even this approach had some challenges as follows:

- Client-side caching increased operational complexity as engineers had to maintain consistency between client and server caches.
- Refresh operations strained the database, especially during updates or restarts when caches had to be reloaded.

Full Refresh-Ahead Cache

To overcome some of the challenges with client-side caching, the team adopted a fu refresh-ahead cache model for the system.

In this approach, each client stored all restriction data in its local memory, eliminat the need for frequent database queries. A polling mechanism regularly checked for updates to maintain cache freshness.

This led to a remarkable improvement in latencies, primarily because all member d was readily available on the client side. There was no need for network calls.

However, the approach also had some limitations and trade-offs:

- The memory footprint was significant, as each client needed enough memory t store the entire dataset.
- During application restarts or deployments, the system experienced high resou consumption to reload all data, resulting in performance bottlenecks.
- The increased database load during refresh operations led to latency spikes and stressed the Oracle database infrastructure.

Bloom Filters

To address scalability and efficiency challenges, LinkedIn implemented Bloom Filte

Bloom Filter is a probabilistic data structure designed to handle large datasets efficiently. Instead of storing the full dataset, it used a compact, memory-efficient encoding to determine whether a restriction record existed in the system. If a query matched a record in the Bloom Filter, the system would proceed to apply the restriction.

The main advantage of using Bloom Filter was to conserve valuable resources. It reduced the memory footprint compared to traditional caching mechanisms. Also, queries were processed rapidly, improving system responsiveness.

However, Bloom Filters also had some trade-offs:

- Bloom Filters are probabilistic, meaning there is a small chance of false positiv where the filter incorrectly indicates a restriction exists.
- Despite this, the trade-off was deemed acceptable for LinkedIn's use case, as it ensured high performance and scalability.

Second Generation

As LinkedIn grew and its platform became more complex, the engineering team recognized the limitations of its first-generation system, which relied heavily on relational databases and caching strategies.

To keep up with the demands of a billion-member platform, the second generation LinkedIn's restriction enforcement system was designed.

The second-generation system had several important goals such as:

- **High QPS (4-5 million):** The system had to handle restriction enforcement for every request across LinkedIn's extensive product offerings.
- **Ultra-Low Latency (<5 ms):** The new system relied on in-memory lookups for fetching restriction data, eliminating the need for repeated database queries. T approach drastically reduced response times, ensuring a seamless experience fo LinkedIn members and applications.
- **High Availability (Five 9's):** The system was designed to achieve 99.999% availability, a level of reliability critical for enforcing restrictions without interruptions. By distributing data across multiple nodes and data centers, the system minimized the risk of downtime.
- **Horizontal Scaling:** To support the growing volume of restrictions and request the system could scale horizontally by adding more nodes or servers.

- **Data Freshness and Synchronization:** Real-time updates through Kafka ensure that all restriction data remained synchronized across the platform, avoiding inconsistencies.

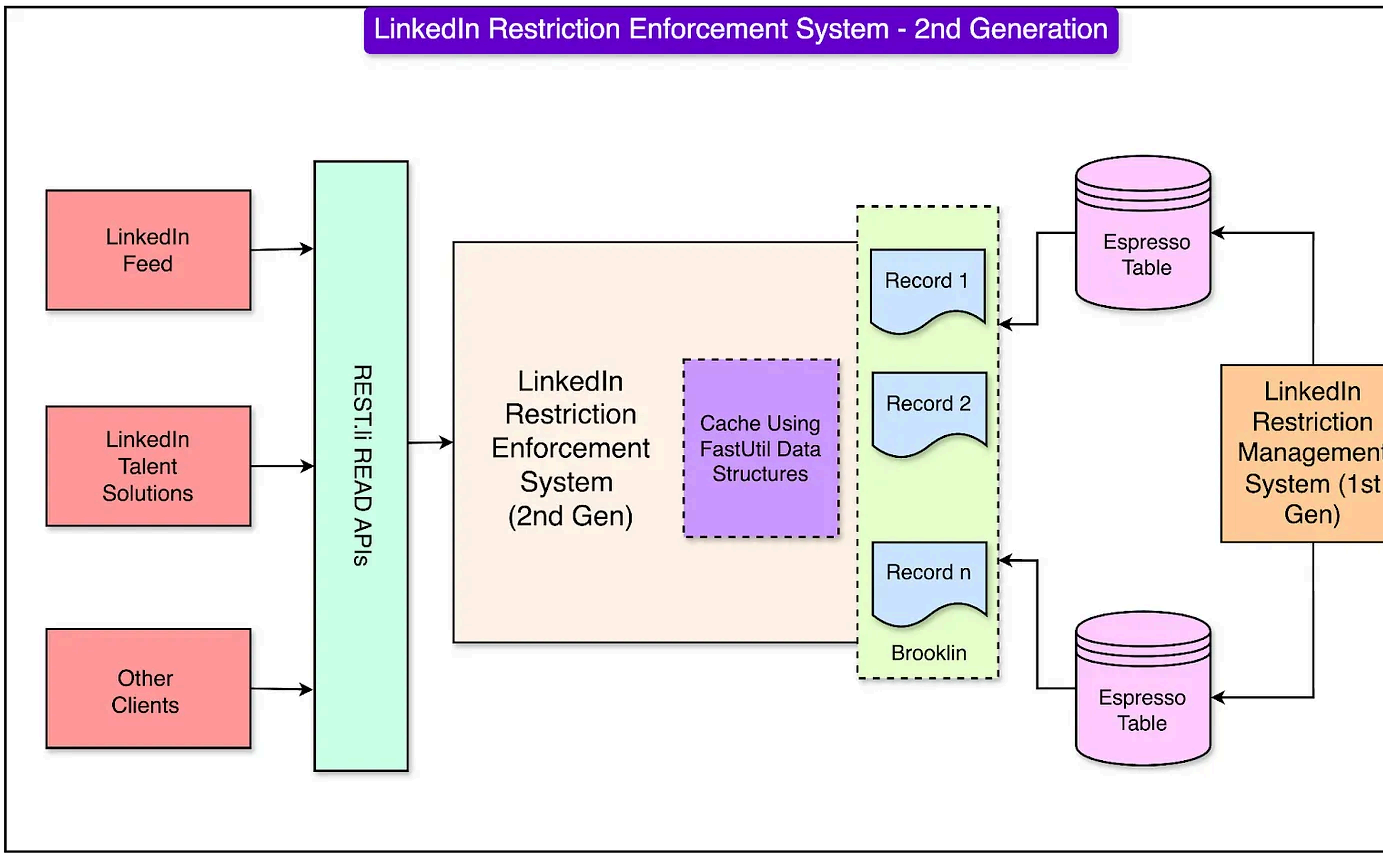
Adoption of NoSQL Distributed Systems

One of the key innovations in this generation was the migration of restriction data management to LinkedIn's Espresso, a custom-built distributed NoSQL document store.

This is because relational databases like Oracle struggled with the high query throughput and latency requirements of LinkedIn's growing platform. Espresso, be a distributed NoSQL system, provided better scalability and performance while maintaining data consistency.

Espresso was tightly integrated with Kafka, LinkedIn's real-time data streaming platform. Every time a new restriction record was created, Espresso would emit Kaf messages containing the data and metadata of the record. These Kafka messages enabled real-time synchronization of restriction data across multiple servers and da centers, ensuring that the system always had the latest information.

See the diagram below to understand the architecture of the 2nd generation restrict enforcement system.



Despite its many advancements, the second-generation system faced some operatio challenges, particularly during specific scenarios:

- **Bootstrapping Data During Restarts**:
  - When a server or application restarted, it needed to reload all restriction records from the database into its memory.
  - This process, known as bootstrapping, was resource-intensive and time-consuming, often taking over 30 minutes for large datasets.
  - The high load during bootstrapping could strain the Espresso database and impact system performance.
- **Handling Large-Scale Growth**: While the system scaled well horizontally, the sheer volume of restriction records and requests during periods of high adversarial activity tested the limits of infrastructure.

CAP Theorem

LinkedIn had to make critical architectural choices concerning CAP Theorem trade offs while designing the second-generation system.

The CAP Theorem states that a system can only achieve two out of the following th guarantees at any given time:

- **Consistency** (**C**): Every read receives the most recent write or an error.
- **Availability** (**A**): Every request receives a non-error response, regardless of the state of any individual node.
- **Partition Tolerance** (**P**): The system continues to operate even when there is a network partition.

LinkedIn prioritized consistency (C) and availability (A) over partition tolerance (P). The decision was driven by their latency and reliability goals.

The restriction enforcement system needed to provide accurate and up-to-date data across the platform. Incorrect or outdated restriction records could lead to security lapses or poor user experiences. Also, high availability was essential for ensuring that restrictions could be enforced seamlessly, even during peak activity periods.

LinkedIn's previous experiences with partitioned databases revealed that partitions could introduce latencies that conflicted with their stringent performance requirements (for example, ultra-low latency of <5 ms).

The use of Espresso allowed LinkedIn to handle consistency and availability more effectively within their system design. Integration with Kafka ensured that restriction records were synchronized across servers in real-time, maintaining consistency without significant delays.

Third Generation

As LinkedIn grew even further, the second-generation restriction enforcement system, though robust, began to show strain under the increasing volume of data and adversarial attacks.

Therefore, the LinkedIn engineering team implemented a new generation of its restriction enforcement system. See the diagram below:

The third generation introduced innovations focused on optimizing memory usage, improving resilience, and accelerating the bootstrap process.

1 - Off-Heap Memory Utilization

One of the major bottlenecks in the second-generation system was the reliance on in-heap memory for data storage. This approach led to challenges with Garbage Collection (GC) cycles, which caused latency spikes and degraded system performance.

To address these issues, the third-generation system moved data storage to off-heap memory.

Unlike in-heap memory (managed by the Java Virtual Machine), off-heap memory exists outside the JVM's control. By shifting data storage to off-heap memory, the system reduced the frequency and intensity of GC events.

Some benefits of this approach were as follows:

- Off-heap memory provided more space for storing restriction data without overloading the JVM heap.
- This change reduced GC interruptions, resulting in smoother and more consist system performance.
- Hosts could handle larger datasets without increasing the risk of hitting memo limits.

2 - Venice and DaVinci Framework

To further optimize the system, the LinkedIn engineering team introduced DaVinc an advanced client library, and integrated it with Venice, a scalable derived data platform.

Here's how these tools work together:

- DaVinci operates as an eager cache, meaning it proactively loads all restriction data into memory at the start. This eliminated the need for frequent on-deman lookups.
- The restriction data was stored in bitset-like data structures, which are highly memory-efficient. These structures allowed the system to manage large dataset while minimizing memory footprint.
- Venice, a distributed platform designed for derived data, enabled seamless integration and synchronization of restriction data. It allowed DaVinci to fetch and store data efficiently, ensuring high-speed performance even during perioc of intense activity.

The innovations in the third-generation system addressed many of the limitations o its predecessors. A couple of benefits were as follows:

- **Faster Bootstrapping Processes**: With DaVinci's eager caching, restriction dat was loaded into memory more quickly during server restarts or deployments. T

reduced downtime and ensured the system could respond to restrictions almos immediately after initialization.

- **Greater Resilience**: The system was better equipped to handle organic growth (more users and data) and adversarial data growth (spikes in restrictions due to malicious activity). By leveraging memory-efficient data structures and off-hea storage, the system could scale without running into performance bottlenecks.

Conclusion

From the early reliance on relational databases to the adoption of advanced NoSQL systems like Espresso, and the integration of cutting-edge frameworks like DaVinc and Venice, every stage of development of the restriction enforcement system showcased LinkedIn's focus on innovation.

However, this journey was not just about innovation. It was also guided by a clear s of principles such as:

- **Start Simple, Scale Thoughtfully:** Early designs avoided unnecessary complexi focusing on solving immediate problems in a straightforward manner.
- **Proactive Problem Identification:** This approach allowed the engineering tean anticipate challenges, such as memory pressure or latency spikes, and address them with strategic solutions.
- **Collaboration Across Teams:** Cross-team collaboration enhanced efficiency by promoting knowledge sharing and reducing redundant efforts.
- **Benchmarking and Testing:** Time-bound experiments and performance benchmarks allowed the team to evaluate approaches quickly.
- **Continuous Improvement:** Each generation of the restriction enforcement syst is built upon the successes and shortcomings of the previous one.

References:

- The Evolution of Enforcing Our Professional Community Policies at Scale
- Building Trust and Combating Abuse on our Platform

156 Likes · 12 Restacks

Discussion about this post

Comments Restacks

OO Write a comment...

Fearless Donkey Fearless's Substack Jan 21, 2025

"LinkedIn prioritized consistency (C) and availability (A) over partition tolerance (P). The decis was driven by their latency and reliability goals."

Please correct me if I misunderstand anything --

I don't think this is an option. As long as it is a distributed system, network partition can't be avoided. In the context of distributed system, CAP basically mean, when partition happens, a system can choose either high consistency or high availability.

Or does LinkedIn not actually using any distributed storage/cache?

LIKE (2) REPLY

3 replies

3 more comments...